

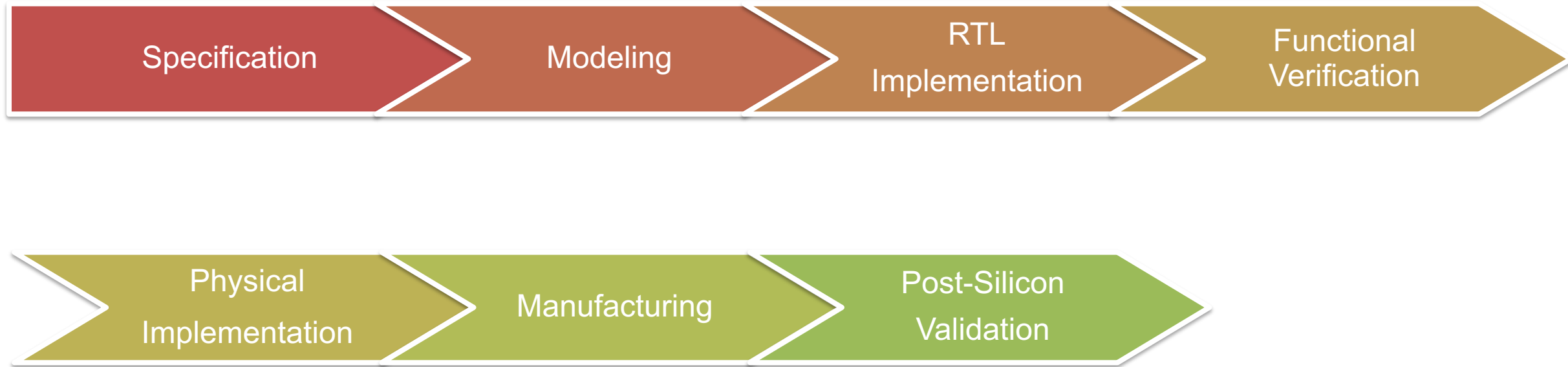
Verification & DFT

UCSD ECE 260C Spring 2026

Davit Markarian

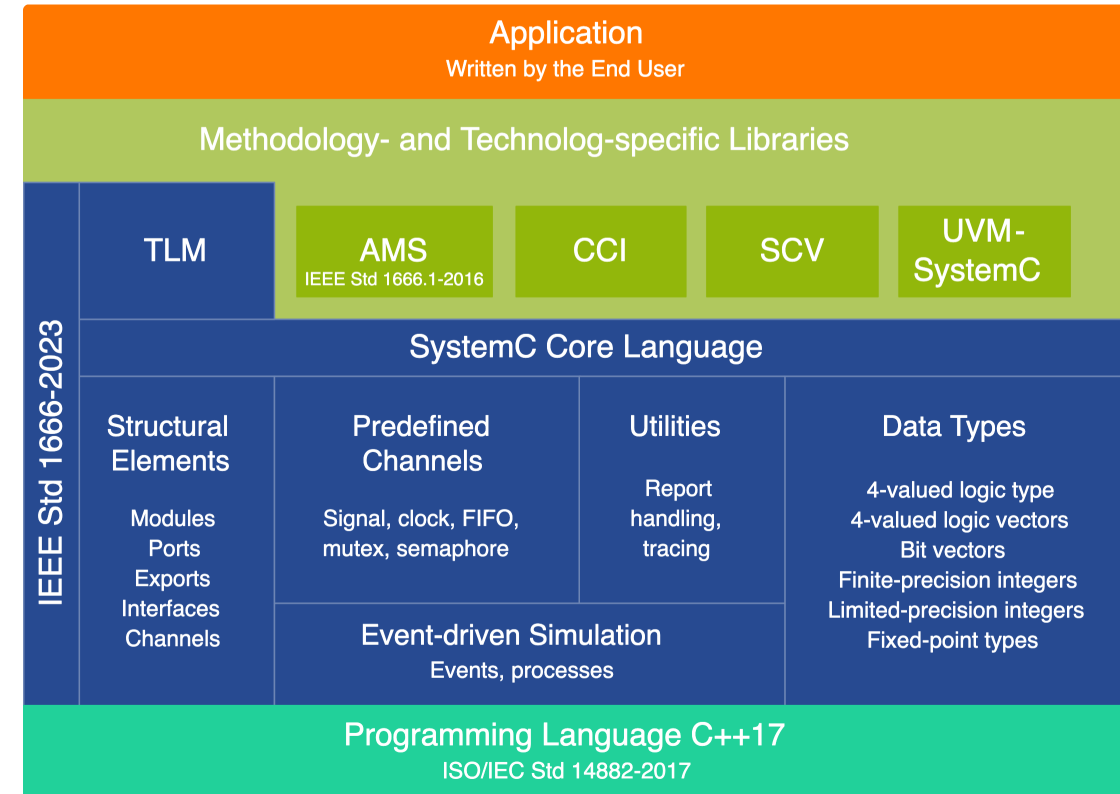
dtmar@ucsd.edu

The ASIC Design Flow

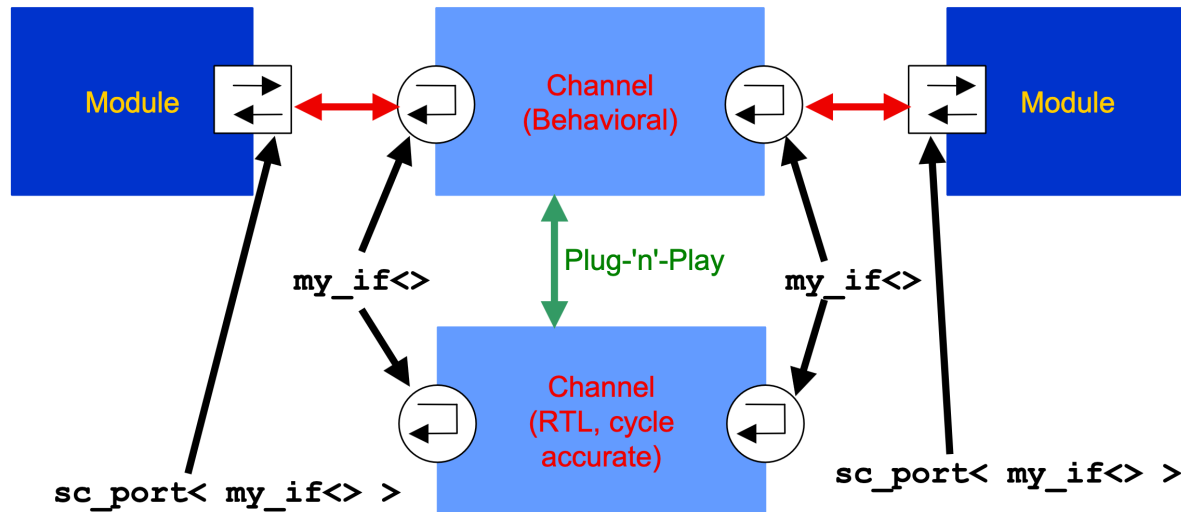


Modeling

- Writing RTL is costly. How can we more quickly validate potential architectures or produce testcases?
- Write models at a higher abstraction !
 - Pipelines
 - Channels (Buses and synchronized ports)
 - Processes (like always@ blocks)
- Do it in C++. Do it in Python...
- Or... [SystemC](#)
 - A modeling library on top of C++
 - Supported by many simulators
 - (can I just [synthesize](#) my [models](#)?)



SystemC Channels



```
template <class T>
class sc_fifo : public sc_fifo_in_if<T>,
               public sc_fifo_out_if<T>,
               public sc_prim_channel
{
    // Interface methods
    virtual void register_port(sc_port_base&, const char*);

    // Read (blocking/nonblocking)
    virtual void read(T&, int);
    virtual T read(int);
    virtual bool nb_read(T&);

    // Capacity and status
    virtual int num_available() const;
    virtual const sc_event& data_written_event() const;

    // Write (blocking and non-blocking)
    virtual void write(const T&, int);
    virtual bool nb_write(const T&);

    // Free space and status
    virtual int num_free() const;
    virtual const sc_event& data_read_event() const;
}
```

Simulation-based Verification

- What you already know: write a testbench, run a simulator
- [Verilator](#) – fast, open, (cycle-driven) simulator
 - Write SystemVerilog or C++ testbenches, then connect to DUT
 - [sv-tests](#) repo tracks open-source SystemVerilog support
- C++-based testbenches offers many possibilities
 - Visual or interactive simulation harnesses
 - SystemC and C++ models
 - Very high speed execution
 - Co-simulation: simulated DUT + real hardware ([Renode](#))
 - [SystemVerilog DPI](#)

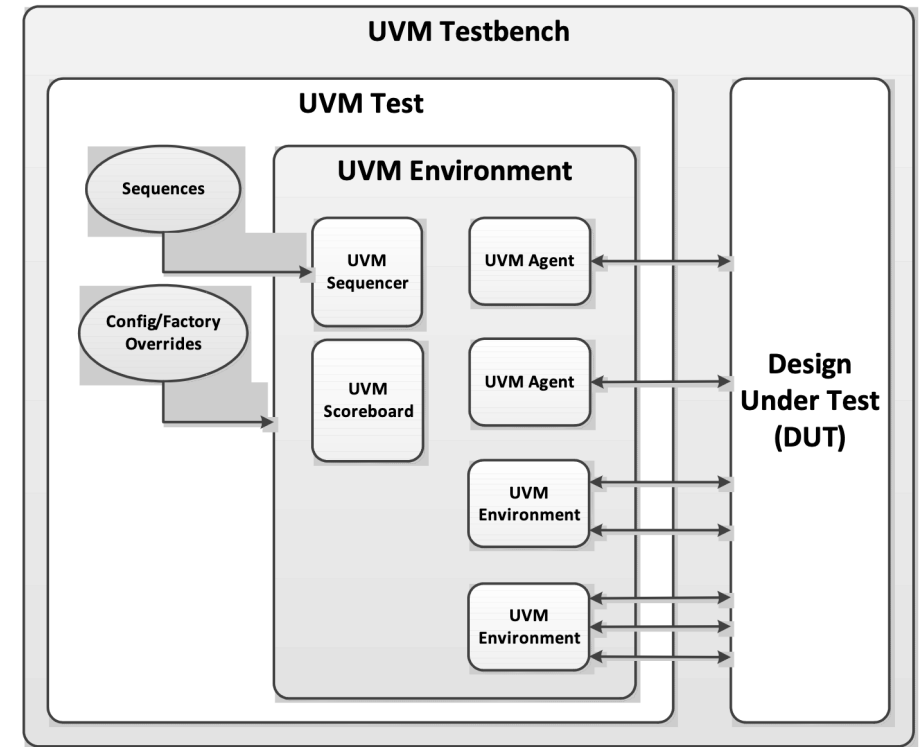
CIRCT and Arcilator

- CIRCT is an LLVM project that extends the LLVM compiler and MLIR generic intermediate representation format to hardware.
 - Goal of creating a rich, unifying intermediate format for logic representation
- This SSA form can be processed through multiple passes, which change depending on task/tool
- Arcilator is a simulator using CIRCT/LLVM
 - It *lowers* CIRCT to its own dataflow IR
 - Then, it optimizes and lowers again to LLVM IR
 - LLVM performs an optimizing compile to native code (2-4x faster than Verilator)

```
hw.module private @Fibonacci(  
  in %clock : !seq.clock,  
  in %rst : i1,  
  in %en : i1,  
  out out : i32,  
  out count : i32)  
{  
  %c1_i32 = hw.constant 1 : i32  
  %c0_i32 = hw.constant 0 : i32  
  %0 = comb.mux %rst, %c0_i32, %5 : i32  
  %reg_0 = seq.compreg %0, %clock : i32  
  %1 = comb.mux %rst, %c1_i32, %3 : i32  
  %reg_1 = seq.compreg %1, %clock : i32  
  %2 = comb.mux %rst, %c0_i32, %7 : i32  
  %counter = seq.compreg %2, %clock : i32  
  %3 = comb.mux %en, %reg_0, %reg_1 : i32  
  %4 = comb.add %reg_0, %reg_1 : i32  
  %5 = comb.mux %en, %4, %reg_0 : i32  
  %6 = comb.add %counter, %c1_i32 : i32  
  %7 = comb.mux %en, %6, %counter : i32  
  hw.output %reg_1, %counter : i32, i32
```

Testbench Writing and Orchestration

- SystemVerilog as a Hardware Verification Language
 - Object-Oriented Programming
 - Randomizations
 - Assertions/Coverage
- Universal Verification Methodology (UVM)
 - SV library that formalizes testbench structures
 - Separation of
 - Stimulus generation
 - Testbench driving/monitoring
 - Scoreboarding
 - Transaction-level support
 - [Works in Verilator](#)
 - [Cocotb](#) / [Vunit](#): simulator-agnostic testing frameworks



Assertions & Coverage

- **Assertions:** combinational/sequential properties that should hold true
 - `a_overflow: assert (count <= MAX_DATA);`
 - `a_counts: assert (count == 0 || count == $past(count) + 1);`
- **Functional coverage:** how many (edge)cases did we test?
 - Many metrics: unit tests run, bits flipped, random samples taken, ...
- **In SystemVerilog,**
 - A sampled value can be stored, binned or unbinned, in a `coverpoint`.
 - `covergroup` objects store `coverpoints` and sample their value at an event e.g., `@posedge clk`.

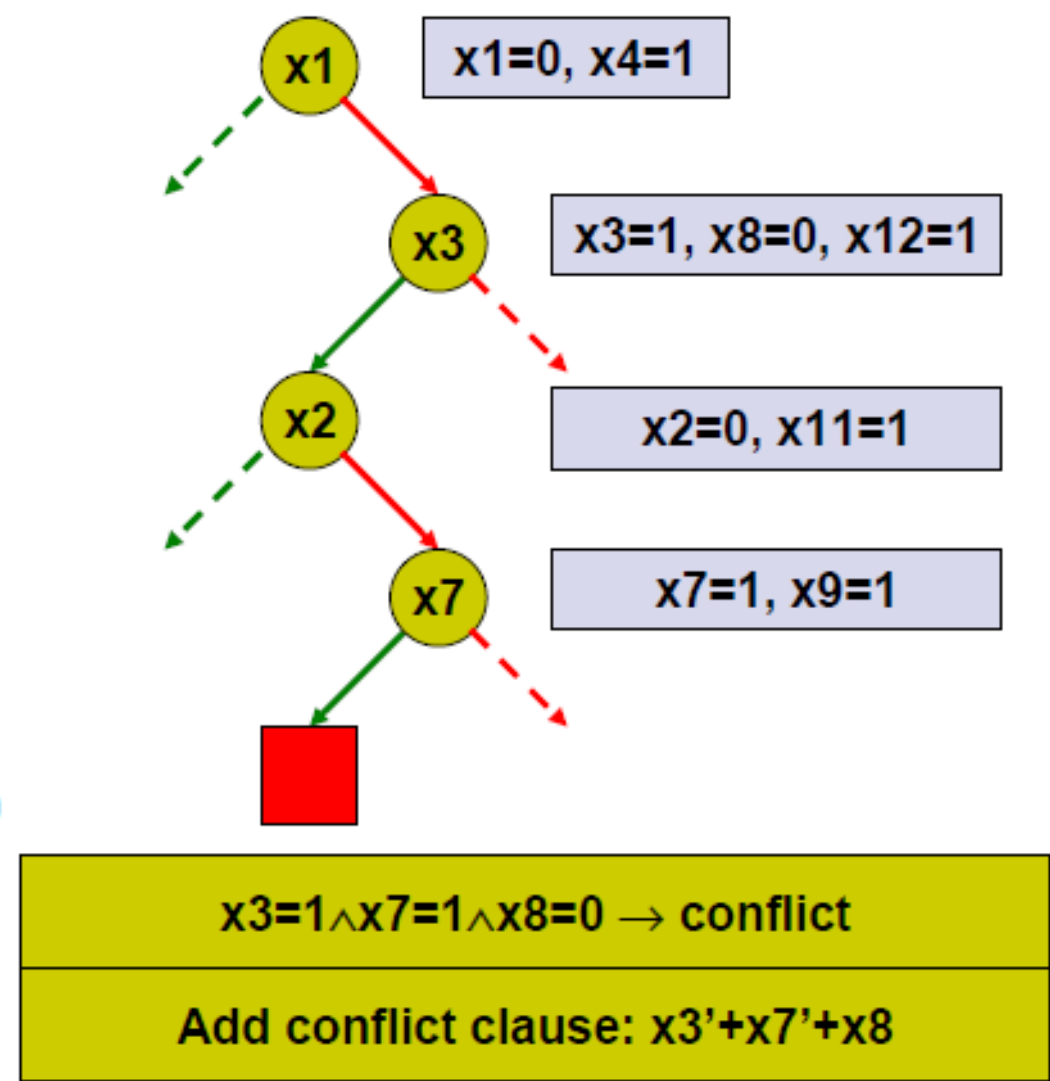
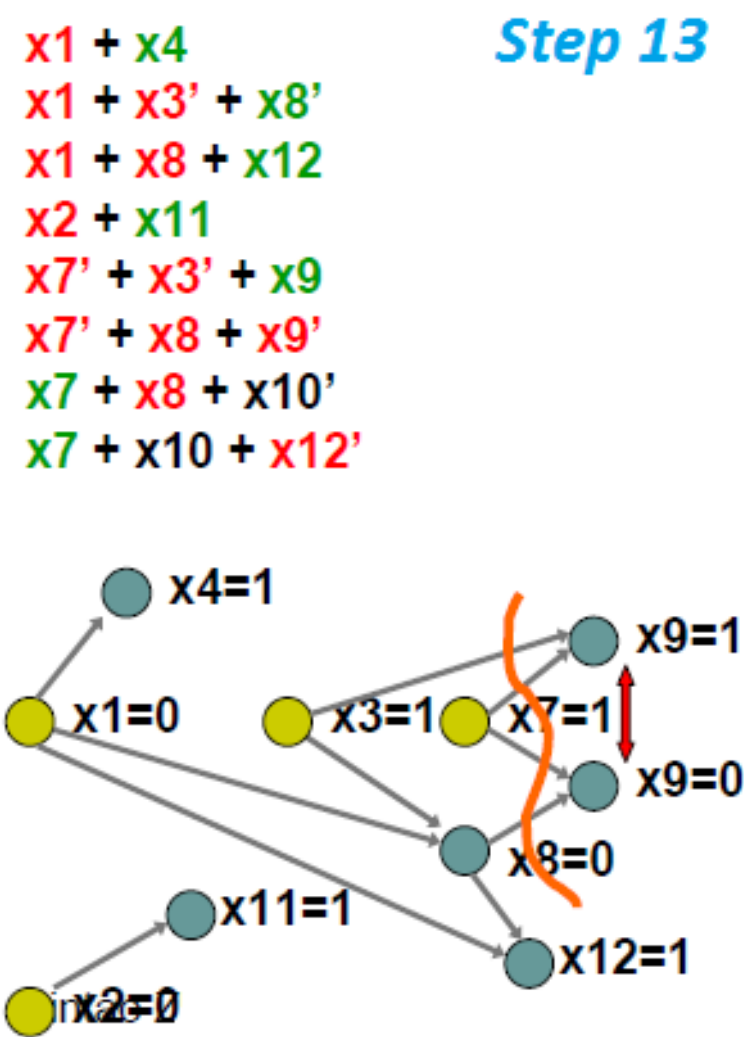
Formal Verification

- Instead of manually creating stimulus to try to break assertions, what if it could be automated?
 - What if this could be done by a guided search?
- Formal Verification uses a prover engine to find counter-examples to your assertions (see: [SymbiYosys](#))
 - Formally verifying full cores is experimental (e.g., [riscv-formal](#)). Smaller structures or modules are fair game.
 - Still sees limited use today
- When verification fails, a minimum viable counterexample is provided – useful for debugging.
- Successful verification provides strong guarantees over your assertions
 - In some sequential cases, proof bounded to “within k cycles”

Boolean SATisfiability & SMT

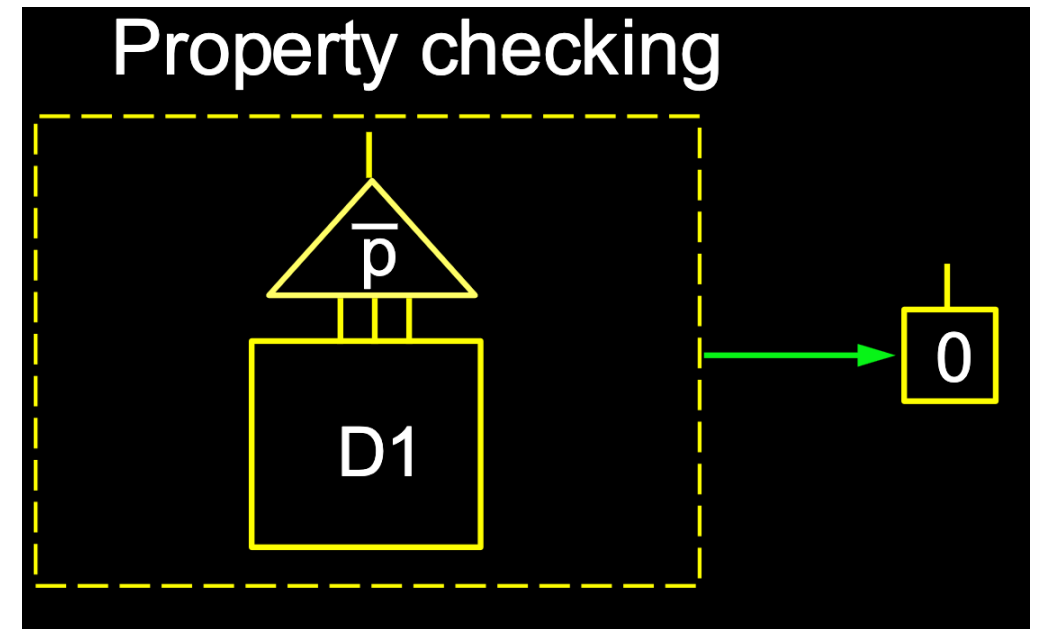
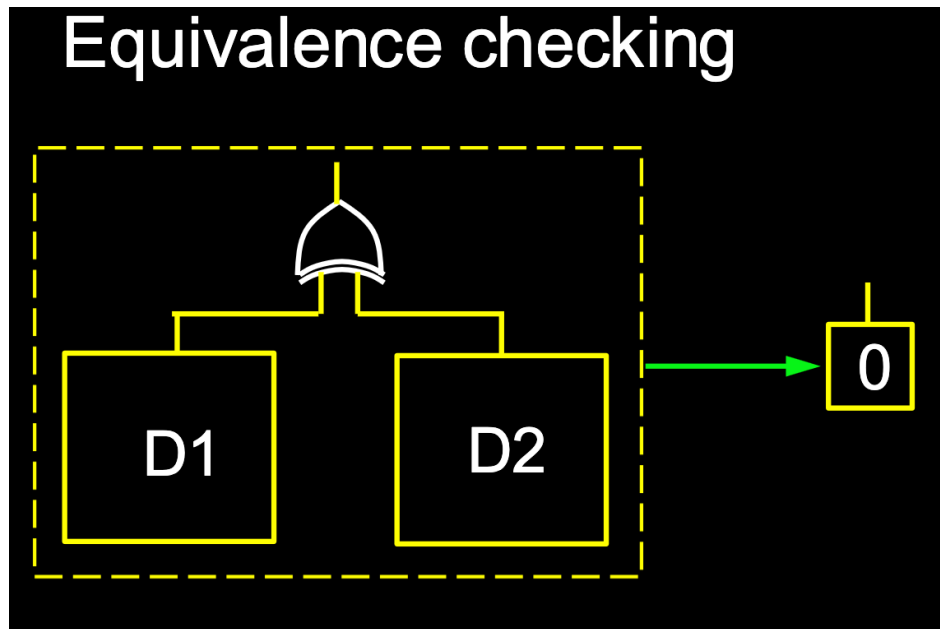
- Underpinning formal verification tools, is a SAT/SMT solver.
- Boolean SAT formulation is in the form of $(x_1 \vee \cdots \vee x_n) \wedge \cdots$
- SAT/SMT solvers are available off-the-shelf (e.g., [MiniSAT](#) or [Z3](#))
 - SMT is a generalization of SAT to e.g., integers or data structures
- SAT/SMT solvers are primarily driven by a search-and-backtracking algorithm called Conflict-Driven Clause Learning (CDCL)
 - A graph of implications is built and known/input constraints are propagated.
 - Conflicts (two opposing assignments to the same variable) are detected and are "learned" by adding additional clauses to the graph and backtracking

CDCL Example



Logical Miter

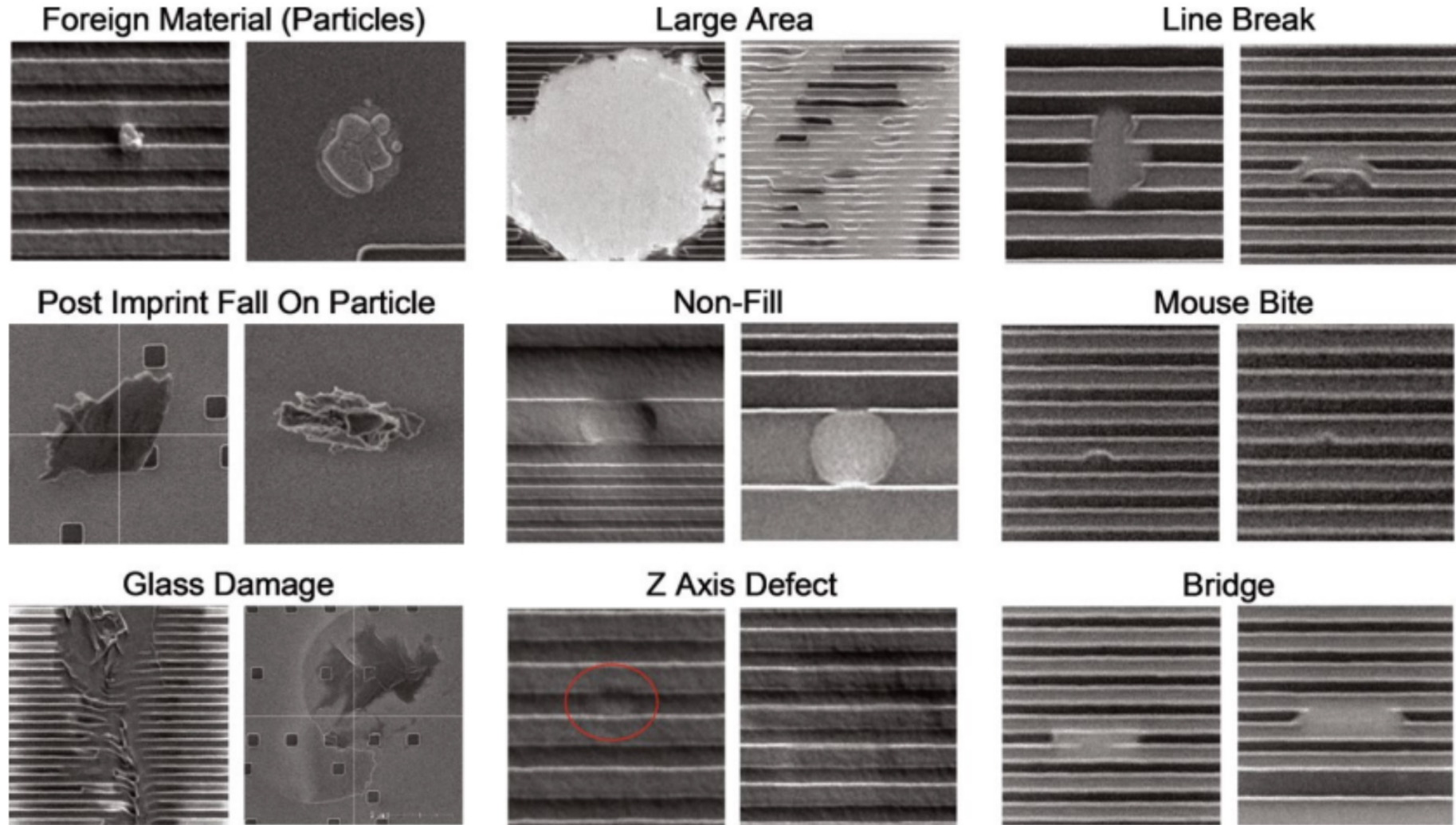
- Logic must be converted into a form acceptable for SMT solver
- ... but first, the assertion we want to check must be encoded
- A miter can be created and optimized by a synthesis tool e.g., yosys/abc
- With the miter model, the SAT formulation looks for points where the property checking assertion fails



Post-Silicon Validation

- Are we done after tapeout? No!
 - How do we test that a chip is working as intended? **Scan chains**
 - Is the packaging working as intended? **Boundary Scan**
 - Can we predict failure behaviors? **Fault simulation**
 - Can the chip run at intended speed? **at-speed** testing
 - Do we have to reject the chip or can we repair it? **BIST/BISR**
-
- Design For Test (DFT) !

What is a fault?

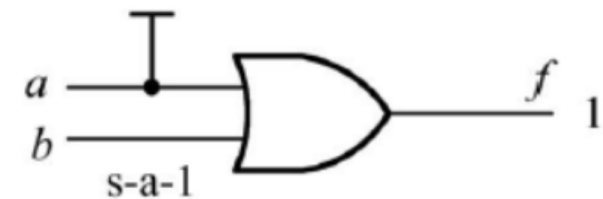


Faults and Fault Modeling

- We want to maximize our fault coverage (# faults vs. faults detected)
- How can we abstract faults? Can we create test vectors to look for them?
- Wire shorts to P/G → Stuck-At-0 or Stuck-At-1 faults
- Two signal wires are bridged → Wired AND/OR
- Transistor faults → stuck-open, stuck-short
 - Wires can be left floating. Does this affect observability?
- Timing/Crosstalk faults: do we observe these at-speed?
- Automated Test Pattern Generation (ATPG)
 - We cannot test all possible input vectors. What is the minimum subset?



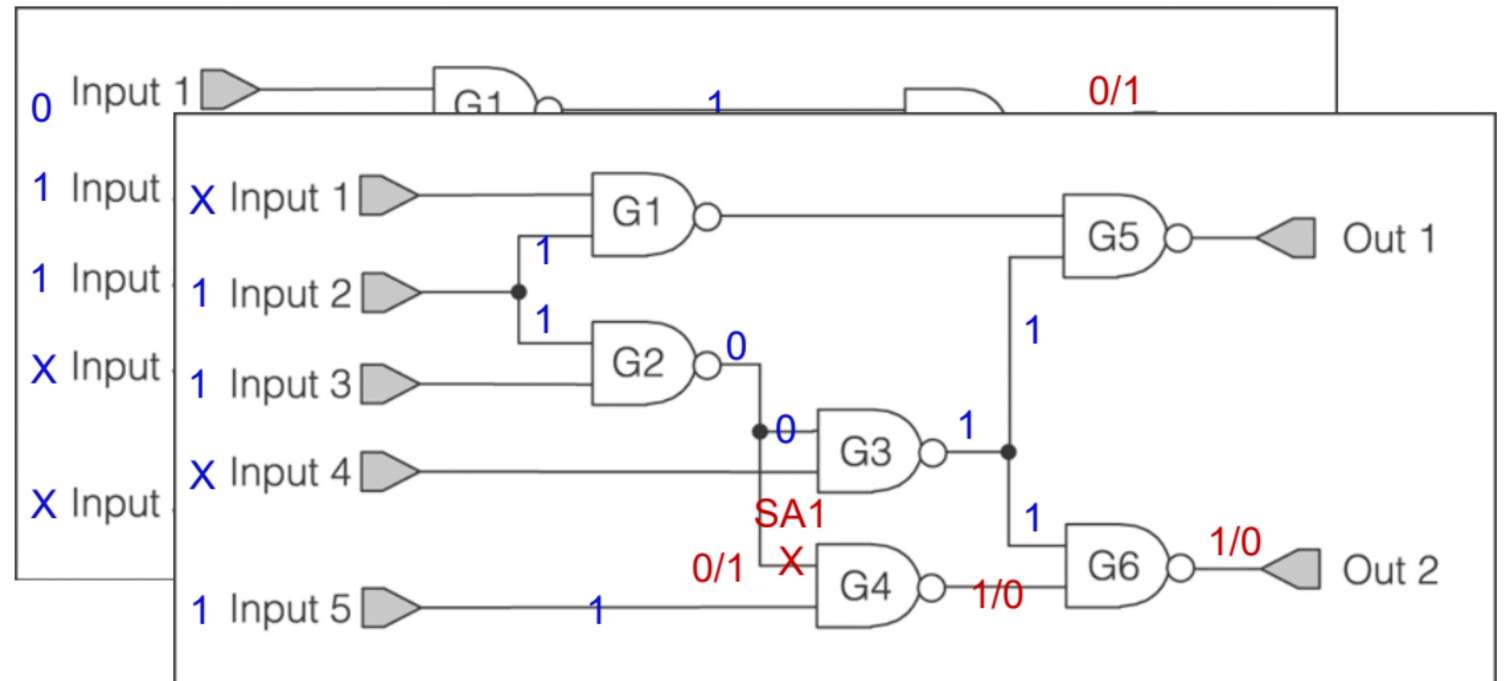
(a) Stuck-at-0 fault



(b) Stuck-at-1 fault

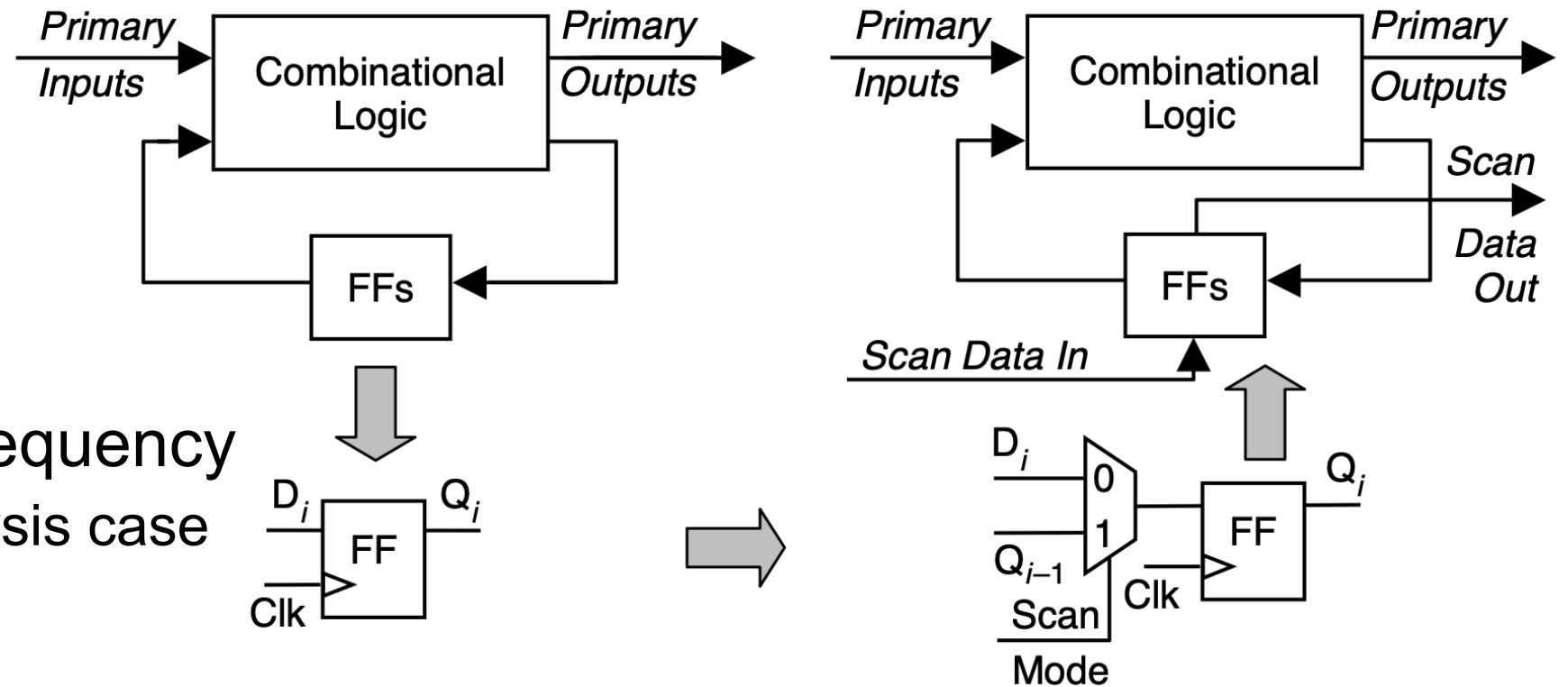
ATPG Test Generation

- We want to merge as many tests as possible to speed up test
- Layout-aware ATPG
 - We know that distant wires are unlikely to short
 - We know when wire is shared – $(0,1,1,X,X) + (X,1,1,X,1) \rightarrow (0,1,1,X,1)$
 - The more X's, the better chances to merge (Test time reduction)



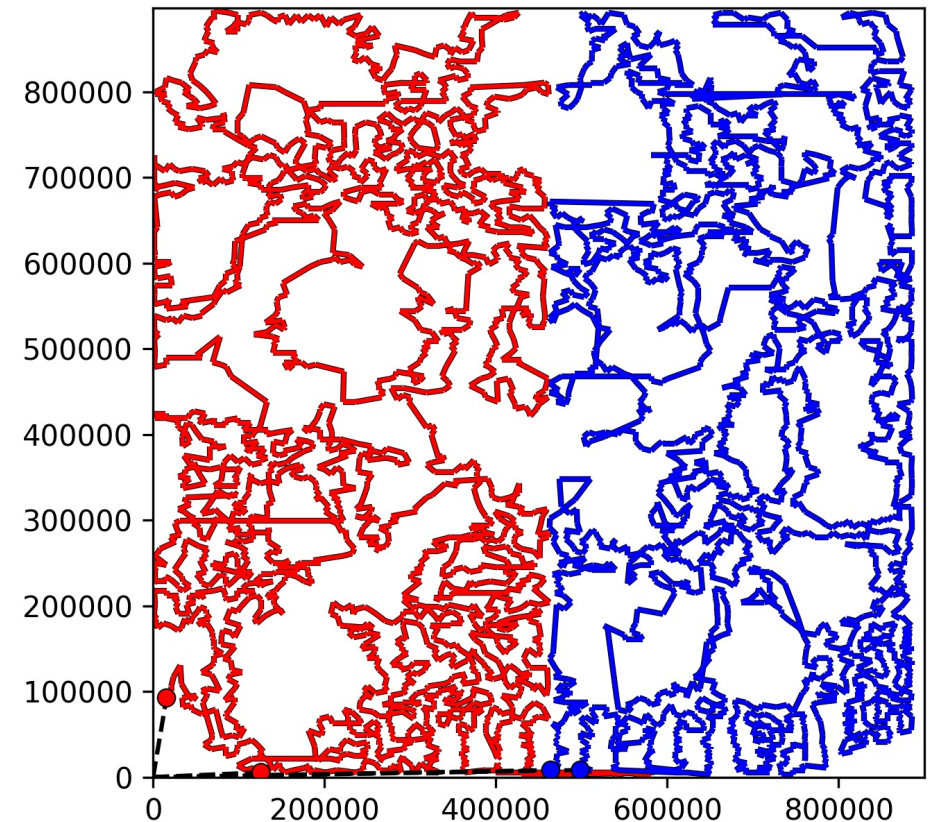
DFT: Implementing Scan Chains

- We “upgrade” flops into scan flops (DFF + MUX)
- To create a scan chain, we connect the output of each flop into the Scan-In of another flop.
- Typically ~100s flops
- Connect to TAP (Test Access Port)
 - Compression/MUXing is possible
- Reduced clock frequency
 - Different STA analysis case



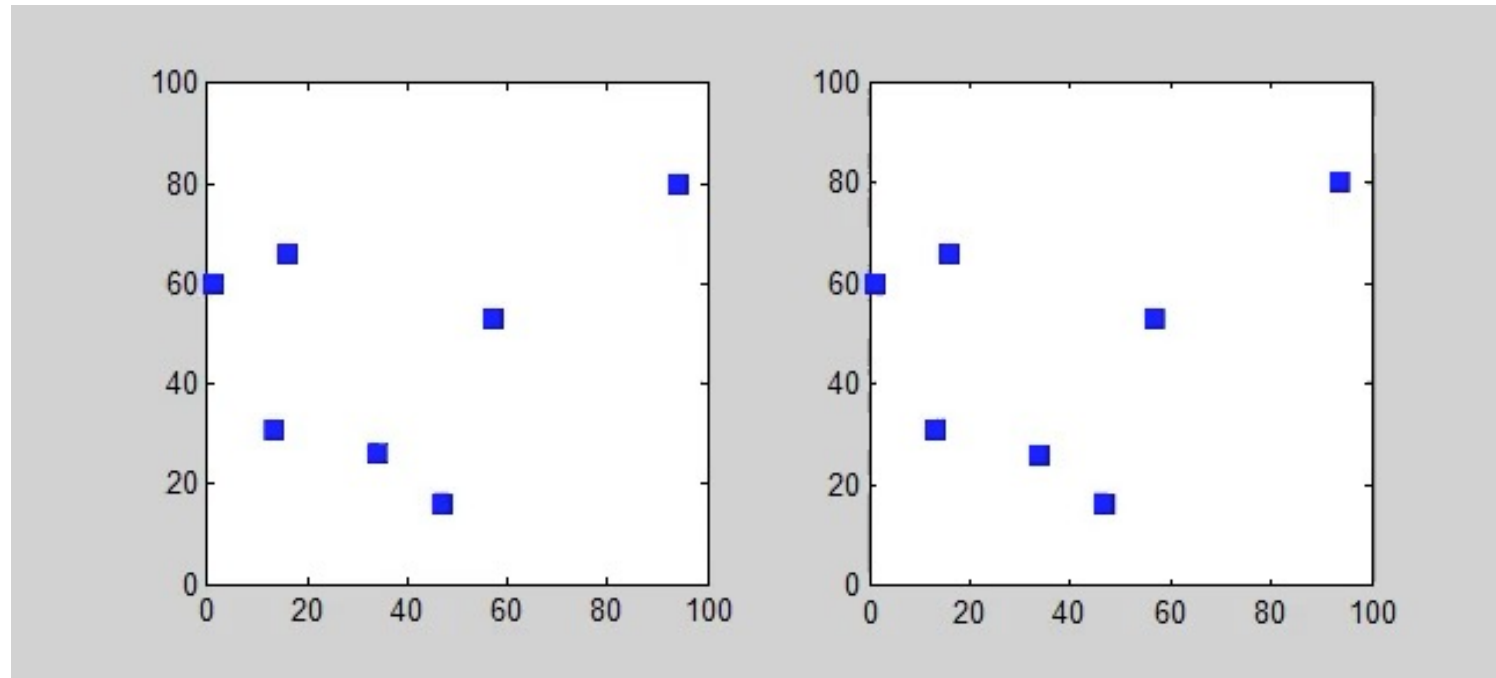
Physical Scan Chain Optimization

- For a partition, test architects typically provide # of chains, max length of the chain, and groupings
 - Flops driven by different edges or clock domains should not share a chain
- How do we select chains?
 - k-means clustering + exchange
- How do we minimize wirelength?
 - How do we maximize scan speed?
 - Traveling Salesman Problem
- Routing and timing impacts?
 - Additional fanout on every flop output!
 - Additional routing between flops
- In [OpenROAD](#)



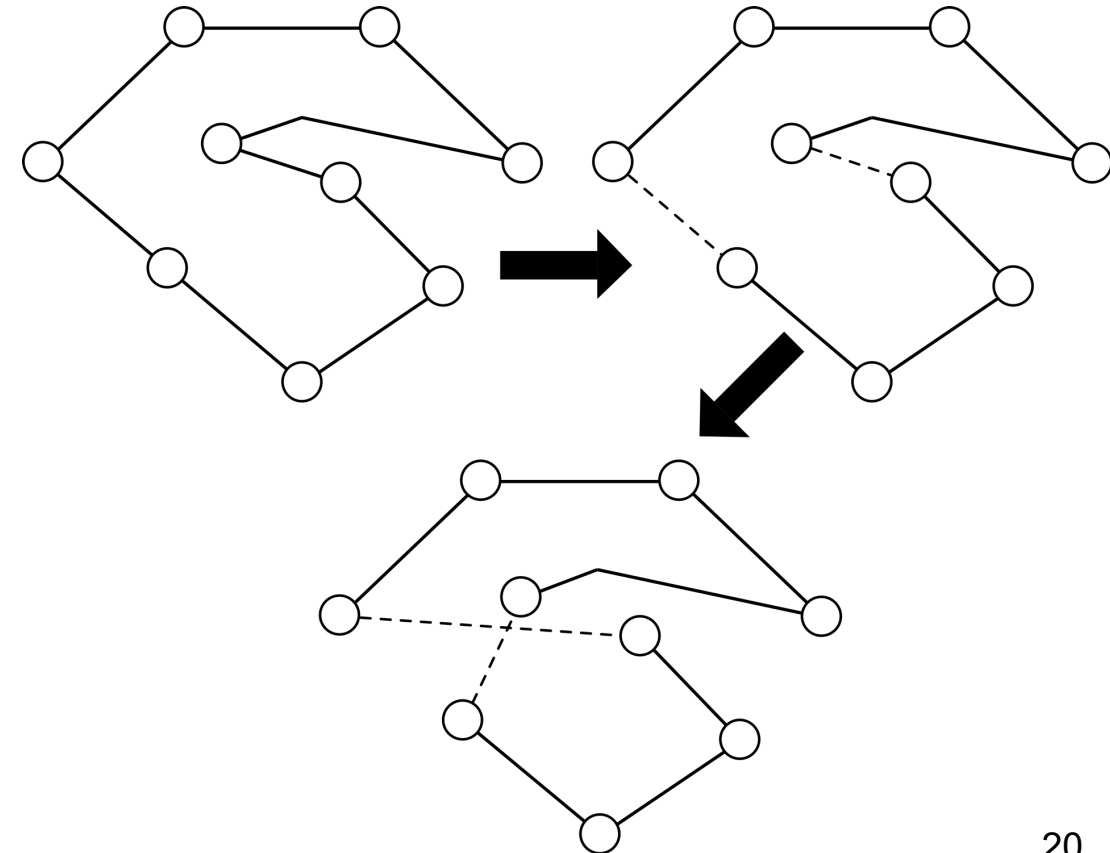
Traveling Salesman Problem

- **Formulation:** Given n cities, create a tour – a path with no city visited twice
 - In DFT, the TSP is asymmetric: we “enter” a flop at its Scan-In and leave at its Q
- **NP-Hard:** Exact algorithms struggle to scale (Held-Karp is $O(2^n n^2)$)
- Solvers: [UCLA ScanOpt](#) and [Google OR-Tools](#)



Heuristic

- In UCLA ScanOpt and OR-Tools, Heuristic TSP is employed
- **k-Opt**: Delete k edges and insert k new ones.
- **Or-Opt**: take a contiguous k-subset and shift it up/down the chain
- Apply metaheuristics (e.g., SA)
 - Need to escape local minima
 - ScanOpt does greedy descent.
OR-Tools supports many solvers.



Built-In Self-Test/Recovery

- Logic with scan chains can be connected to on-chip test instead of test generators
 - Applications for high lifespan for detecting failures due to thermal, EM
- BIST/BISR sees most use in SRAM
 - Memory tests ensure that values can be changed and are retained
 - Executed by BIST Controller logic
 - Recovery is possible by MUXing addressing to use redundant spare columns